Лабораторная работа № 22: Исследование методов подключения к серверу MySQL и работы с БД.

Цель работы: Сформировать умение анализировать методы подключения к серверу MySQL и работы с БД.

---

## Задание 1 (Базовый уровень)

Установка соединения с MySQL и выполнение простого запроса

Описание:
Необходимо установить соединение с сервером MySQL и выполнить простой SELECT-запрос. Продемонстрировать разницу между одиночным соединением и пулом.

Как это должно работать:

1. Настройка подключения:
    - Параметры подключения берутся из переменных окружения (`.env`)
    - Хост: `localhost`, порт: `3306`
    - Пользователь: `student` (не root!)
    - База данных: `bbmo_01_23` (создаётся отдельно)
    - Пароль хранится в `.env`
2. Одиночное соединение (`createConnection`):
    - Создаётся одно TCP-соединение
    - Выполняется запрос `SELECT VERSION()`
    - Соединение закрывается после выполнения
    - В консоль выводится версия MySQL
3. Пул соединений (`createPool`):
    - Создаётся пул из 10 соединений
    - Выполняется тот же запрос
    - Соединение возвращается в пул автоматически
    - Пул остаётся открытым для дальнейших запросов
4. Проверка подключения:
    - Вывод информации о сервере: версия, текущая БД, пользователь
    - Обработка ошибок подключения (`ECONNREFUSED`, `ER_ACCESS_DENIED_ERROR`)

Пример работы:

```
text
```

```
$ node mysql-connect.js

=== Одиночное соединение ===
✔ Подключение установлено
Версия MySQL: 8.0.35
Текущая БД: bbmo_01_23
Пользователь: student@localhost
✖ Соединение закрыто

=== Пул соединений ===
✔ Пул создан (лимит: 10)
Версия MySQL: 8.0.35
Текущая БД: bbmo_01_23
Соединений в пуле: 10

✔ Пул готов к работе
```

Что нужно сделать для отчёта:

- Сделать скриншот успешного одиночного подключения
- Сделать скриншот успешного подключения через пул
- Сделать скриншот ошибки подключения (неверный пароль)
- Сделать скриншот содержимого `.env` (без пароля)

---

## Задание 2 (Средний уровень)

Создание таблицы и выполнение CRUD-операций

Описание:
Создать таблицу `students` и выполнить базовые CRUD-операции через пул соединений.

Как это должно работать:

1. Создание таблицы:
    - Таблица `students` со столбцами:
        - `id` — INT, PRIMARY KEY, AUTO_INCREMENT
        - `name` — VARCHAR(100), NOT NULL
        - `group_name` — VARCHAR(20), NOT NULL
        - `course` — INT, NOT NULL

        - `grade` — DECIMAL(3,2)
        - `created_at` — TIMESTAMP, DEFAULT CURRENT_TIMESTAMP
    - Движок: InnoDB, кодировка: utf8mb4
2. INSERT (Create):
    - Вставка одного студента через `INSERT`
    - Проверка `insertId` и `affectedRows`
    - Вставка нескольких студентов через `INSERT ... VALUES (...), (...)`
    - Группа студентов — ББМО-01-23
3. SELECT (Read):
    - Выборка всех студентов
    - Выборка с фильтром по группе
    - Выборка с сортировкой по оценке
    - Выборка с `LIMIT` и `OFFSET` (пагинация)
4. UPDATE:
    - Обновление оценки студента
    - Проверка `affectedRows` и `changedRows`
    - Демонстрация: `affectedRows=1, changedRows=0` при обновлении тем же значением
5. DELETE:
    - Удаление студента по ID
    - Проверка `affectedRows`
    - Удаление всех студентов группы (осторожно!)
6. Параметризованные запросы:
    - Все запросы с данными пользователя — через `?` (плейсхолдеры)
    - Демонстрация SQL-инъекции при конкатенации строк
    - Демонстрация защиты через плейсхолдеры

Пример работы:

```
text
```

```
$ node mysql-crud.js

=== Создание таблицы ===
✔ Таблица students создана

=== INSERT ===
✔ Вставлен студент: id=1, affectedRows=1
✔ Вставлено 3 студента: affectedRows=3

=== SELECT ===
Все студенты:
  id=1, name=Иван, group=ББМО-01-23, grade=4.50
```

```
  id=2, name=Мария, group=ББМО-01-23, grade=3.80
  id=3, name=Петр, group=ББМО-01-23, grade=4.20

Студенты группы ББМО-01-23 (сортировка по grade):
  id=1, name=Иван, grade=4.50
  id=3, name=Петр, grade=4.20
  id=2, name=Мария, grade=3.80

=== UPDATE ===
✔ Обновлён студент id=1: affectedRows=1, changedRows=1
✔ Повторное обновление: affectedRows=1, changedRows=0

=== DELETE ===
✔ Удалён студент id=1: affectedRows=1

=== Защита от SQL-инъекции ===
Попытка инъекции: "'; DROP TABLE students; --"
✔ Параметризованный запрос безопасен

✖ Конкатенация привела бы к удалению таблицы
```

Что нужно сделать для отчёта:

- Скриншот создания таблицы
- Скриншот INSERT с `insertId` и `affectedRows`
- Скриншот SELECT с фильтром и сортировкой
- Скриншот UPDATE с разницей `affectedRows` и `changedRows`
- Скриншот DELETE
- Скриншот демонстрации SQL-инъекции

---

## Задание 3 (Продвинутый уровень)

Обработка ошибок и работа с потоковыми запросами

Описание:
Изучить категории ошибок MySQL и реализовать потоковую обработку больших наборов данных.

Как это должно работать:

1. Категории ошибок:
    - Сетевые: `ECONNREFUSED`, `ETIMEDOUT`, `PROTOCOL_CONNECTION_LOST`

    - Аутентификация: `ER_ACCESS_DENIED_ERROR`
    - Синтаксис: `ER_PARSE_ERROR`
    - Конфликты: `ER_DUP_ENTRY` (дублирование уникального ключа)
    - Ограничения: `ER_NO_REFERENCED_ROW` (нарушение внешнего ключа)
2. Демонстрация каждой ошибки:
    - Попытка подключения к несуществующему хосту
    - Попытка входа с неверным паролем
    - Синтаксически неверный SQL
    - Вставка дубликата по уникальному ключу
    - Попытка вставить NULL в NOT NULL столбец
3. Обработка ошибок пула:
    - Слушатель `error` на самом пуле
    - Логирование глобальных ошибок
    - Пересоздание пула при критических ошибках
4. Потоковые запросы (Streaming):
    - Генерация 100 000 записей в таблицу `big_data`
    - Выполнение `SELECT * FROM big_data`
    - Использование `.stream()` для чтения по кускам
    - Обработка через события `data`, `end`, `error`
    - Сравнение памяти: `SELECT` целиком vs `.stream()`
5. Транзакции:
    - Начало транзакции через `pool.getConnection()`
    - `connection.beginTransaction()`
    - Выполнение нескольких INSERT
    - `connection.commit()` или `connection.rollback()`
    - Обязательный `connection.release()` в `finally`

Пример работы:

```
text

$ node mysql-errors.js

=== Демонстрация ошибок ===
[ECONNREFUSED] Сервер недоступен: localhost:9999
[ER_ACCESS_DENIED_ERROR] Неверный пароль для пользователя student
[ER_PARSE_ERROR] Синтаксическая ошибка: SELECT * FRM students
[ER_DUP_ENTRY] Дублирование ключа: id=1
[ER_BAD_NULL_ERROR] NULL в NOT NULL столбце: name

=== Обработка ошибок пула ===
[POOL ERROR] PROTOCOL_CONNECTION_LOST
[POOL] Пересоздание пула...
```

```
=== Потоковый запрос ===
Чтение 100000 записей...
[STREAM] Получено: 10000 (10%)
[STREAM] Получено: 50000 (50%)
[STREAM] Получено: 100000 (100%)
✔ Обработано за 2.3 сек
Пик памяти: 45 МБ

Сравнение: SELECT целиком
✖ Ошибка: JavaScript heap out of memory
  Требуется ~500 МБ памяти

=== Транзакция ===
Начало транзакции
✔ INSERT 1 выполнен
✔ INSERT 2 выполнен
✖ INSERT 3 не выполнен (дубликат)
Rollback выполнен

Соединение возвращено в пул
```

Что нужно сделать для отчёта:

- Скриншот каждой категории ошибок
- Скриншот обработки ошибок пула
- Скриншот потокового чтения
- Скриншот сравнения памяти
- Скриншот успешной транзакции
- Скриншот отката транзакции

---

## Задание 4 (Сложный уровень)

Полноценный слой доступа к данным (DAO) с валидацией

Описание:
Создать слой доступа к данным (Data Access Object) для работы с таблицей `students`, включающий валидацию, пагинацию, поиск и статистику.

Как это должно работать:

1. Класс `StudentDAO`:
    - Инкапсулирует все SQL-запросы

    - Использует пул соединений
    - Не раскрывает SQL-синтаксис наружу
    - Все методы — асинхронные (Promise)
2. Методы класса:
    - `create(data)` — создание студента с валидацией
    - `findById(id)` — получение по ID
    - `findAll(options)` — список с пагинацией, сортировкой, фильтрами
    - `update(id, data)` — обновление
    - `delete(id)` — удаление
    - `search(query)` — поиск по имени
    - `getStats()` — статистика (средний балл, количество по группам)
3. Валидация:
    - `name` — обязательное, от 2 до 100 символов
    - `group_name` — обязательное, формат `ББМО-XX-XX`
    - `course` — целое, от 1 до 4
    - `grade` — от 0 до 5, может быть NULL
4. Пагинация:
    - `limit` (по умолчанию 10)
    - `offset` (по умолчанию 0)
    - Возврат общего количества записей
    - Возврат метаданных: текущая страница, всего страниц
5. Фильтры:
    - По группе (`group_name`)
    - По курсу (`course`)
    - По диапазону оценок (`grade_min`, `grade_max`)
    - Комбинирование фильтров
6. Сортировка:
    - По имени (A-Z, Z-A)
    - По оценке (возрастание, убывание)
    - По дате создания
7. Статистика:
    - Общее количество студентов
    - Средний балл
    - Количество по группам
    - Количество по курсам
    - Минимальная и максимальная оценка

Пример работы:

```
text
```

```
$ node mysql-dao.js
```

```
=== StudentDAO (группа ББМО-01-23) ===

--- Создание с валидацией ---
✔ Создан: id=1, name=Иван
✖ Ошибка: name слишком короткое (мин. 2 символа)
✖ Ошибка: group_name не соответствует формату ББМО-XX-XX

--- Пагинация ---
findAll({ limit: 5, offset: 0 }):
  Всего: 100, Страница: 1/20
  id=1, name=Иван, group=ББМО-01-23, grade=4.50
  id=2, name=Мария, group=ББМО-01-23, grade=3.80
  ...

--- Фильтры ---
findAll({ group_name: 'ББМО-01-23', course: 2 }):
  Найдено: 25 студентов

findAll({ grade_min: 4.0, grade_max: 5.0 }):
  Найдено: 42 студента

--- Поиск ---
search('Иван'):
  Найдено: 3 студента
  id=1, name=Иван Иванов
  id=15, name=Иван Петров
  id=42, name=Иван Сидоров

--- Статистика ---
Всего студентов:       100
Средний балл:          4.12
Минимальная оценка:    2.50
Максимальная оценка:   5.00

По группам:
  ББМО-01-23: 40
  ББМО-02-23: 35
  ББМО-03-23: 25

По курсам:
  1 курс: 30
  2 курс: 35
  3 курс: 20
  4 курс: 15
```

Что нужно сделать для отчёта:

- Скриншот структуры класса `StudentDAO`
- Скриншот создания с валидацией
- Скриншот пагинации
- Скриншот фильтров
- Скриншот поиска
- Скриншот статистики

---

## Задание 5 (Экспертный уровень)

Микросервис на Koa.js с MySQL: REST API с транзакциями и кэшированием

Описание:
Создать полноценный REST API на Koa.js с использованием MySQL, включающий транзакции, кэширование, миграции и мониторинг.

Как это должно работать:

1. Структура проекта:
    - `config/` — конфигурация БД, переменные окружения
    - `models/` — модели данных
    - `services/` — бизнес-логика
    - `controllers/` — контроллеры
    - `middleware/` — middleware (БД, ошибки, логирование)
    - `migrations/` — миграции БД
    - `routes/` — маршруты
2. Middleware для БД:
    - Внедрение пула в `ctx.db` на раннем этапе
    - Все контроллеры используют `ctx.db`
    - Автоматическое освобождение соединений
3. REST API для студентов:
    - `GET /api/students` — список с пагинацией
    - `GET /api/students/:id` — один студент
    - `POST /api/students` — создание
    - `PUT /api/students/:id` — обновление
    - `DELETE /api/students/:id` — удаление
    - `GET /api/students/stats` — статистика
4. REST API для групп:
    - `GET /api/groups` — список групп

    - `POST /api/groups` — создание группы
    - `GET /api/groups/:id/students` — студенты группы
5. Транзакции:
    - `POST /api/students/batch` — массовое создание
    - При ошибке одного INSERT — откат всех
    - Использование `connection.beginTransaction()`
    - Обязательный `connection.release()` в `finally`
6. Кэширование:
    - In-memory кэш для GET-запросов
    - TTL: 60 секунд
    - Инвалидация при изменении данных
    - `GET /api/cache/stats` — статистика кэша
7. Миграции:
    - Таблица `migrations` для отслеживания
    - Файлы миграций: `001_create_students.sql`, `002_create_groups.sql`
    - Команда `npm run migrate` для применения
    - Команда `npm run migrate:rollback` для отката
8. Мониторинг:
    - Логирование всех SQL-запросов
    - Метрики: количество запросов, среднее время, ошибки
    - `GET /api/metrics` — метрики БД
9. Обработка ошибок:
    - Централизованный middleware
    - Маппинг ошибок MySQL в HTTP-статусы:
        - `ER_DUP_ENTRY` → 409 Conflict
        - `ER_NO_REFERENCED_ROW` → 400 Bad Request
        - `ECONNREFUSED` → 503 Service Unavailable
    - Структурированный JSON-ответ
10. Безопасность:
    - Только параметризованные запросы
    - Валидация входных данных
    - Переменные окружения для секретов
    - Ограничение размера тела запроса

Пример работы:

```
text

$ npm start
[INFO] Пул MySQL создан (лимит: 10)
[INFO] Миграции применены: 2
[INFO] Koa-сервер запущен на порту 3000
```

```
[INFO] Группа: ББМО-01-23

$ curl http://localhost:3000/api/students?limit=3
{
  "data": [
    {"id": 1, "name": "Иван", "group_name": "ББМО-01-23", "grade": 4.5},
    {"id": 2, "name": "Мария", "group_name": "ББМО-01-23", "grade": 3.8},
    {"id": 3, "name": "Петр", "group_name": "ББМО-01-23", "grade": 4.2}
  ],
  "pagination": {"total": 100, "limit": 3, "offset": 0, "pages": 34}
}

$ curl -X POST http://localhost:3000/api/students/batch \
  -H "Content-Type: application/json" \
  -d '[{"name": "Анна", "group_name": "ББМО-01-23", "course": 2, "grade":
4.7},
       {"name": "Олег", "group_name": "ББМО-01-23", "course": 2, "grade":
3.9}]'
{
  "created": 2,
  "transaction": "committed"
}

$ curl http://localhost:3000/api/students/stats
{
  "total": 102,
  "averageGrade": 4.11,
  "byGroup": {"ББМО-01-23": 42, "ББМО-02-23": 35, "ББМО-03-23": 25}
}

$ curl http://localhost:3000/api/metrics
{
  "queries": 1523,
  "avgTime": 2.3,
  "errors": 5,
  "cacheHitRate": 0.65
}

# Логи
[2024-01-15 14:30:25] [ББМО-01-23] SELECT * FROM students LIMIT 3 OFFSET 0
(1.2ms)
[2024-01-15 14:30:28] [ББМО-01-23] BEGIN TRANSACTION
[2024-01-15 14:30:28] [ББМО-01-23] INSERT INTO students ... (0.8ms)
[2024-01-15 14:30:28] [ББМО-01-23] COMMIT
```

Что нужно сделать для отчёта:

- Скриншот структуры проекта
- Скриншот запуска сервера
- Скриншот `GET /api/students` с пагинацией
- Скриншот `POST /api/students/batch` с транзакцией
- Скриншот отката транзакции при ошибке
- Скриншот `GET /api/students/stats`
- Скриншот `GET /api/metrics`
- Скриншот логов SQL-запросов
- Скриншот миграций
- Скриншот работы кэша

---

**Фиксация изменений**

Добавить файлы в Git
Создать коммит
Отправить в удалённый репозиторий
Создать Pull Request